Experiment – 8

Name: Bhavik Pahuja

UID: 24BAI70522

Section: 24AIT_KRG G1

A. Experimental Procedure

Step 1: Implementation

Binary Search,

Subset Sum (Verification and Decision versions),

Traveling Salesman Problem (Brute Force).

The implementation should ensure correctness, proper handling of edge cases, and readability.

Step 1: Implementation

The implementation phase focused on developing three distinct algorithms to observe their behavior across different complexity classes. Each was coded to ensure high readability and robust handling of edge cases.

Binary Search (O(log n)):

- Approach: Implemented using an iterative divide-and-conquer strategy on a pre-sorted array.
- Logic: The algorithm repeatedly identifies the middle element and discards the half of the array that cannot contain the target.
- Edge Cases: Handled scenarios involving empty arrays, single-element arrays, and targets that were either smaller than the minimum or larger than the maximum element.

Subset Sum (Verification and Decision):

- Decision Version (O(2^n)): Developed using recursion with memoization to determine if any combination of numbers in a set adds up to a target value K.

- Verification Version (O(n)): Implemented a simple linear-time function that takes a “candidate subset” and validates if its sum equals K.
- Edge Cases: Addressed inputs where the target K is zero, cases with negative integers (if applicable), and sets where the sum of all elements is still less than the target.

Traveling Salesman Problem (Brute Force - O(n!)):

- Approach: Utilized a permutation-based strategy to calculate the total distance of every possible Hamiltonian cycle in a graph.
- Logic: The implementation iterates through all (n-1)! possible paths, keeping track of the one with the minimum total weight.
- Edge Cases: Handled small graphs (2–3 nodes), disconnected graphs, and weighted edges to ensure the logic remained sound regardless of graph density.

Step 2: Measure Metrics

Record key performance metrics for each execution, including execution time, approximate number of operations, and feasibility status (Completed or Timeout). Maintain a structured log for further analysis.

To analyze the “Factorial Wall” and “Exponential Growth,” performance metrics were logged during execution.

- **Execution Time:** Recorded using high-precision timers (milliseconds) to capture the difference between sub-millisecond search results and multi-second brute-force calculations.
- **Approximate Number of Operations:** Calculated based on the Big O theoretical growth (n! for TSP, 2^n for Subset Sum, and log n for Binary Search) to correlate theoretical complexity with actual CPU work.
- **Feasibility Status: Completed:** Assigned to executions where the algorithm finished within a 30-second window.
- **Timeout:** Assigned to instances where the combinatorial explosion made it impossible for the computer to finish (typically seen in TSP with n > 15).

B. Analysis Tasks (Critical Thinking)

Q1. Why does Binary Search consistently demonstrate efficient performance across large input sizes?

Ans. Binary Search remains efficient even as n grows to millions because of its logarithmic time complexity, O(log n). Unlike Linear Search (O(n)), which inspects every element, Binary Search eliminates half of the search space with every single comparison. To illustrate, if we increase the input size from 1,000 to 1,000,000 (a 1000 times increase), the number of operations only increases from roughly 10 to 20. This slow growth rate makes it highly scalable.

Q2. Explain why Subset Sum is computationally difficult to solve but relatively easy to verify.

Ans. Subset Sum is a classic example of an NP-Complete problem. To solve the decision version, we must potentially explore all 2^n subsets. For n=100, the number of subsets exceeds the number of atoms in the observable universe, making an exhaustive search impossible. Verification is simple. We only need to sum the elements in that subset—a linear process taking O(n) time—and compare it to the target. In computational terms, it is easy to check a proof but very hard to find one.

Q3. Identify the input size at which the Traveling Salesman Problem becomes infeasible and justify the reason.

Ans. For a standard brute-force TSP implementation, the threshold of infeasibility is typically around n = 12 to 15. The complexity is O(n!). While exponential growth (2^n) is fast, factorial growth is catastrophic. At n=13, there are roughly 6.2 billion permutations. By n=20, the permutations (2.4 * 10^18) would take decades to compute on a standard workstation. Even with minor optimizations, the brute-force approach cannot bypass the inherent combinatorial explosion.

Q4. Differentiate between solving a problem and verifying a given solution with appropriate examples.

Ans. Solving a Problem: This involves the search for a solution within a large state space without prior knowledge of the answer.

Example: Find subset with sum = target

Verifying a Solution: This involves checking if a provided answer satisfies all the problem's constraints.

Example: Check if given subset sums to target

Q5. Discuss why NP-Complete problems are considered the most challenging class within NP.

Ans. NP-Complete problems are difficult because:

No known polynomial-time solution exists

If one NP-Complete problem is solved efficiently, all NP problems can be solved efficiently

They require checking an exponential number of possibilities

Examples: Subset Sum (Decision), Traveling Salesman Problem